AN0025- EC NB-IoT SDK Beginner Guidebook_V 1.1

EigenCOMM Wireless Microcontroller

Doc Description

This document gives a brief introduction for EC NB-IoT SDK, including architecture of the SDK and supported features.

目录

1. Introduction

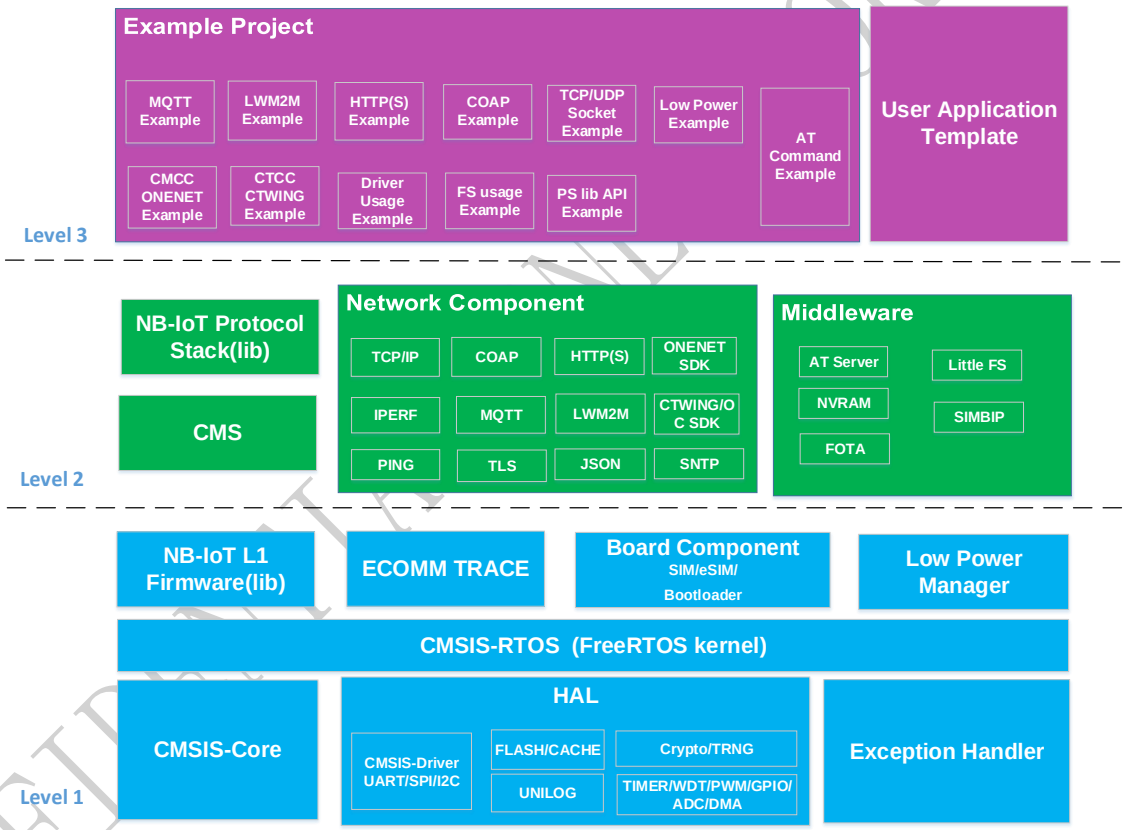
EC NB-IoT SDK provides software and developing tools to help developer who want to develop application software running on EC EC NB-IoT chip. Features supported: peripheral drivers, NB-IoT network access capability , TCP/IP/MQTT/COAP/LWM2M/HTTP(S),Little FS , AT server, FreeRtos and etc.

This document is a guidebook to help developer to understand features included in SDK, architecture of SDK, folder structure of SDK and etc.

1.1 SDK architecture

EC616 SDK includes 3 layers as shown is Figure 1-1.

Figure 1-1. EC616 SDK architecture



Below is a brief introduction for each component.

Level 1 introduction：

- HAL: drivers for all supported peripherals，standard APIs are provided. SPI/I2C/UART drivers APIs follow ARM CMSIS.
- CMSIS Core :ARM CM3 access APIs follow ARM CMSIS CORE, such as APIs to access NVIC, MPU, SYSTICK and etc.
- FreeRTOS: Support FreeRtos kernel, and OS access APIs also follow ARM CMSIS OS.
- ECOMM TRACE: efficient logging API。

- Exception Handler：handler after system crash such as assert/hardfault, all RAM content will be dumped to flash or PC(via debug tool)
- Low Power Manager：low power management APIs.
- Board Component: bootloader ,SIM driver and etc.
- NB-IoT L1 Firmware：NB L1 firmware as a library.

Level 2 introduction：

- Middleware：including middleware porting from third party and developed by EC. Such as: AT server to parse and handle AT command, FOTA, Little FS and etc.
- Network Component：kinds of application layer protocols are supported: TCP/UDP,COAP, LWM2M,HTTP(S),TLS/DTLS,MQTT. Some most commonly used cloud platforms for NB-IoT are also supported, such as CMCC ONENET, CTCC OceanConnect and ali-cloud.
- CMS : Communication Modem Service .APIs to access NB-IoT protocol stack, useful for OPENCPU developing.
- NB-IoT Protocol Stack：both R13 and R14 are supported, share as a library.

Level 3 introduction：

- Example Project：provide demo projects to help developer to getting started with the SDK more efficient. Make developer to understand SDK components more quickly.
- User Application Template：template project to help developer to start their own application.

1.2 Main features of SDK

1.2.1 NB-IoT modem

Table 1-1. Modem features

Modem Features
Fully support 3GPP R14 NB-IoT
Category NB2, 2-HARQ
Multi-tone NPUSCH
Anchor and non-anchor carrier
In-band same/different PCI, guard-band, standalone
Multi-carrier paging, NPRACH
Positioning: OTDOA & ECID
ROHC, RAI, multiple-DRB, RRC connection re-establish
SC-PTM (need SW upgrade)

1.2.2 Network components

Two kinds of network component are supported in EC616 SDK. One for standard application layer network protocol, one for SDKs which support most commonly used IoT device management cloud platforms.

Table 1-2 lists standard application layer network protocol and their basic information.

Table 1-2. Network component-1

Component	features
IP Stack	● IPv4/v6（LWIP） ● TCP, UDP ● DNS ● SOCKET ● SNTP/PING/IPERF
LWM2M	● Eclipse wakaama ● LWM2M Server and Client
MQTT	● eclipse paho MQTT, ● MQTT 3.1.1, ● MQTT client
HTTP(S)	● HTTP1.1 ● Client（POST/GET/PUT/DELETE）
COAP	● Libcoap4.2.0 ● COAP client ● COAPs client ● RFC 7252
TLS	● Mbedtls/tinydtls ● client ● DTLS/TLS1.2
JSON	● cJSON

Table 1-3 lists most commonly used IoT device management cloud platforms.

Table 1-3. IoT device management cloud platforms

Cloud platform	feature
CMCC ONENET	● Access to China Mobile ONENET platform
CMCC DM	● Access to China Mobile device management platform
CTCC CTWING	● Access to China Telecomm Ocean Connect and CTWING platform

1.2.3 Peripheral driver

SDK provides drivers for all peripherals on EC61x SOC.

Table 1-4 lists all drives.

Table 1-4. Drive list

Peripheral	Feature
SPI	● Up to 2 Serial Peripheral Interface ● Master/slave mode
UART	● Up to 3 full feature UART ● Hardware flow control ● Baud rate up to 6M
I2C	● Up to 2 I2C interface
FLASH	● 4MB SiP ● XIP / SW programming ● Erase&programme Suspend/Resume
CACHE	● 16K instruction cache
TIMER	● Up to 6 Timers, ● One shot mode ● Periodic mode ● 6 external IO pulse counters
WDT	● Generate interrupt when expired ● Trigger system reset when expired
PWM	● Up to 6 PWMs
GPIO	● Pull up/down ● Output/input ● 6 AON GPIOs in deep sleep mode
AUXADC	● 4 channel 12 bits AUXADC
UNILOG	● Hardware /software logging
CRYTPO	● Flash encryption ● AES ● SHA1/SHA256
TRNG	● 192 bits truly random number generator

DMA	● 8 channel DMA

1.2.4 Other components

Table 1-5 lists some other components which are not included above.

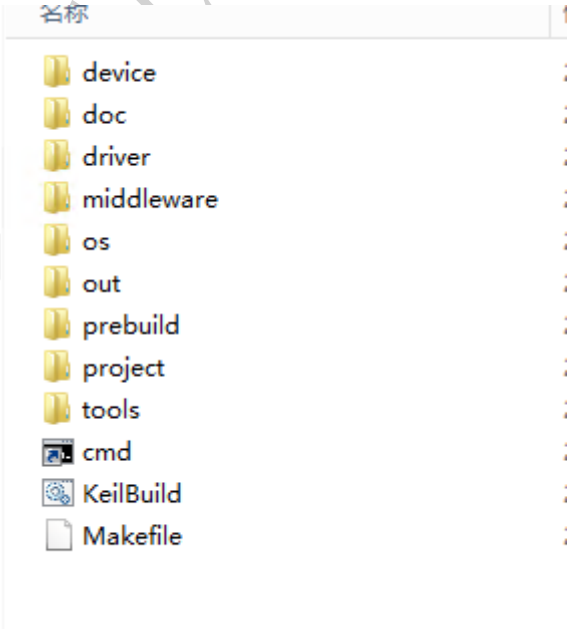
Table 1-5. Other components

component	feature
AT Server	● Parse and handle for both 3GPP at commands and self-defined commands.
LITTLE FS	● Mbed LITTLE FS. high-integrity embedded file system
FOTA	● FOTA via CMCC ONENET and CTCC OceanConnect
SIM BIP	● USIM application toolkit feature，used for transfer data between USIM and remote server

1.3 SDK folder structure

Figure 1-2 shows root folder structure

Figure 1-2. EC616 SDK PLAT root folder



Most files are provided as source file, some components are provided as library in SDK.

1.3.1 folder introduction

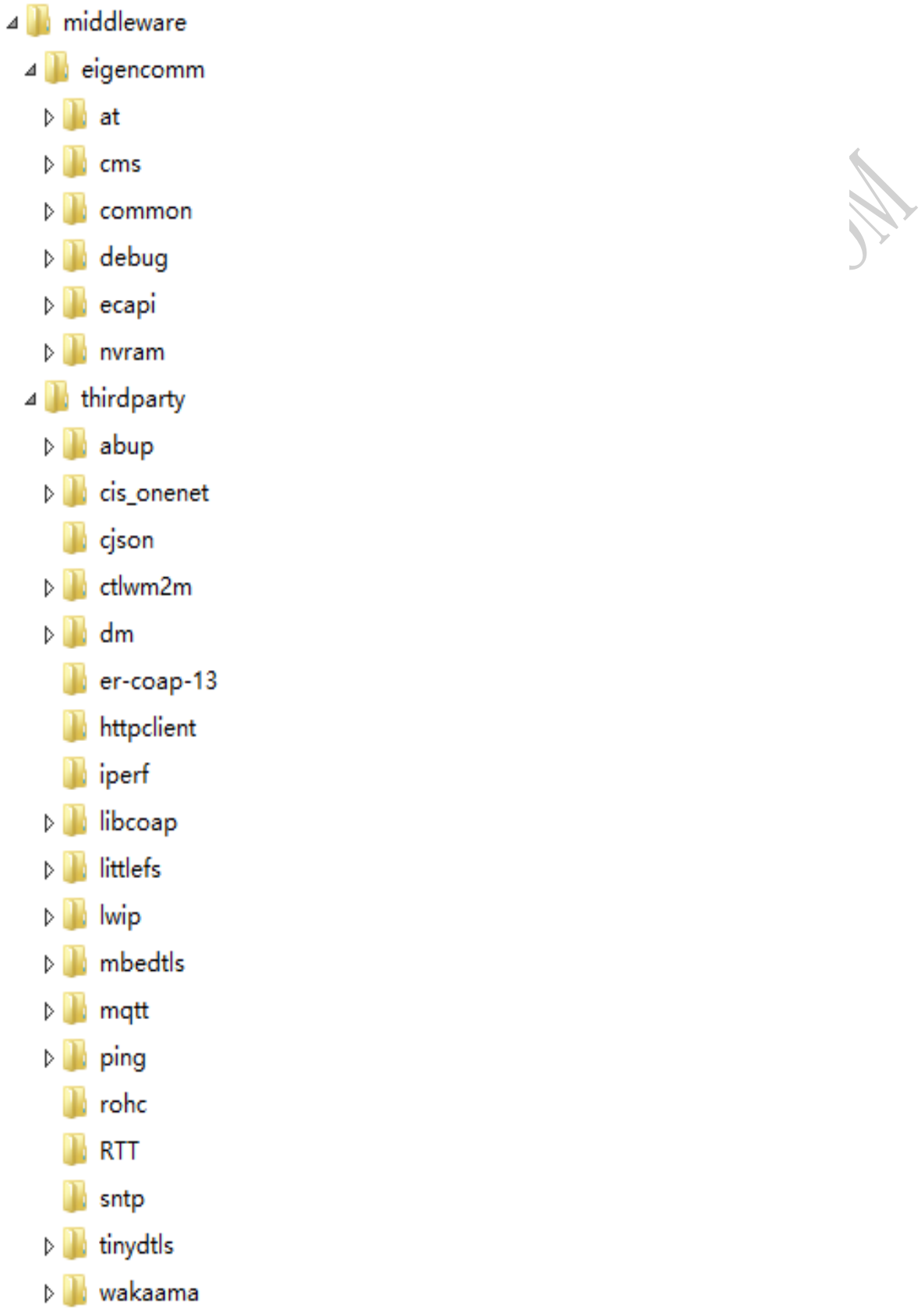
- Device：chip level configures, compile and boot up related files.
- Doc：documents for SDK. Including driver APIs, application notes, compile guidebook and etc.
- Driver： drivers for peripherals
- Middleware：middleware from third party and EC.
- Os：FreeRtos kernel and CMSIS OS adapter APIs.
- Out：output files after successfully compiled.
- Prebuild：libraries for some internal module, such as NB L1 firmware.
- Tools：tools for developing, such as flash download pack for KEIL, LOG pre-parser tool and etc.

1.3.2 Middleware folder introduction

Middleware folder will be introduced separately, including middleware ported from open source module and developed by EC, listed in figure 1-3.

- At：used for parsing and handling AT command.
- Cms：used for interaction with protocol stack.
- Common：common feature such as some configuration.
- Debug：Debug API.
- Ecapi： OpenCPU API
- Nvram： used for reliable data storage.

Figure 1-3. EC616 SDK Project folder

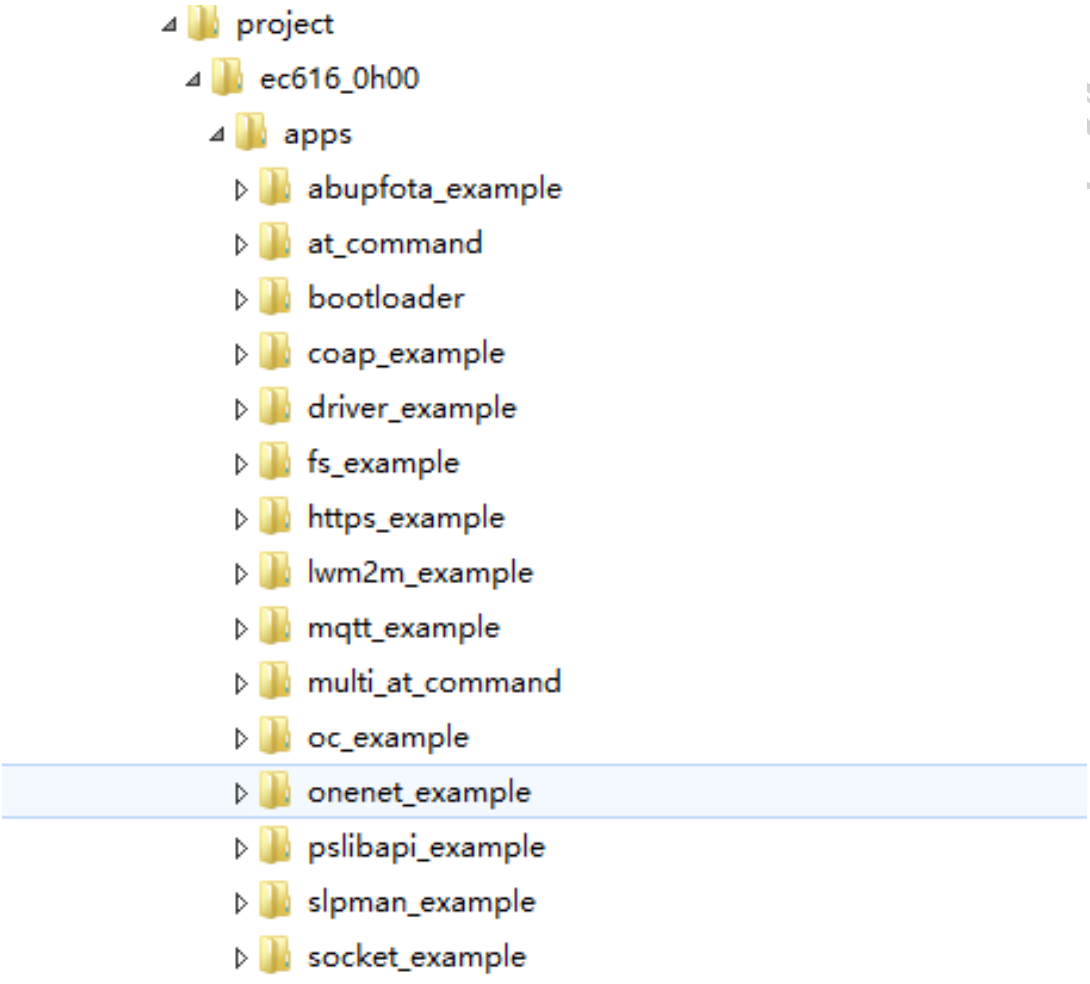


1.3.3 Project folder introduction

Project folder will be introduced separately, since developer usually get started here.

Project：as shown in Figure 1-4, project folder includes all demo projects

Figure 1-4. EC616 SDK Project folder



- Abupfota_example: FOTA example.
- At_command：default project. All application could be used via at command interface.
- Bootloader：only open source for some customers.
- Coap_example：COAP example.
- Driver_example: example for using drivers.
- Fs_exmaple：example for using Little FS.
- https_example: HTTP example.
- lwm2m_example: LWM2M example.
- mqtt_example：MQTT example.
- socket_example：TCP.UDP example.
- pslibapi_example: example for using protocol stack control APIs.

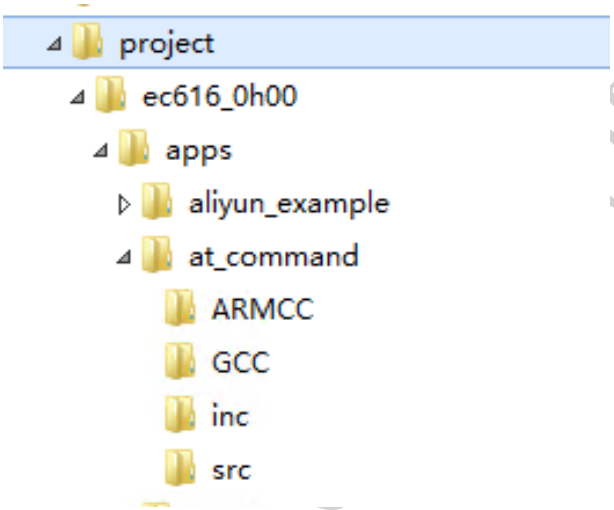
- slpman_example: example for using sleep manager APIs.

Above example projects could be separated into two parts: one for developer to understand some components, e.g. driver example; another one for developer to develop specific applications, since the example is based on specific application protocols, e.g. at_command for developing at command application.

According to whether need AT command or not, the example could also be separated into two parts: one for module, which should be controlled only by at commands; another one for OPENCPU product, developer uses API to develop their application directly.

Take AT command project as an example, the folder structure is shown below:

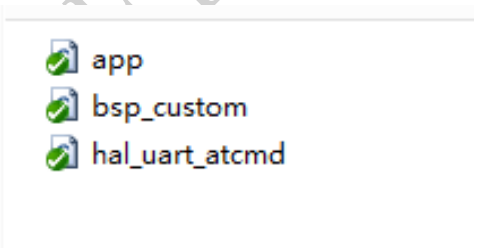
Figure 1-5. SDK Project Folder



- ARMCC: makefile for arm toolchain.
- GCC :makefile for arm toolchain. * To be supported.
- Inc：header files
- Src：source files，shown in figure 1-6.App.c is application entry; bsp_custom.c is customer BSP initialization ; hal_uart_atcmd.c is at command UART initialization.

Source files in src is listed in figure1-6

Figure 1-6. Detail of at_commnad/src



2. Develop with ARM toolchain

This chapter describes how to develop with arm toolchain, and how to download and debug the compiled image. Below topic will be discussed.

1 How to compile EC NB-IoT SDK.

2 How to create your own project

3 How to download and debug

2.1 How to compile SDK

After unpack the SDK, there are several batch files in the root directory. For example, KeilBuild.bat is a One-Key compilation script that supports the ARMCC toolchain.

For more detail, refer to AN0020-EC NB-IoT SDK Compiling Guidebook_V1.0.

2.1.1 Setup ARM Toolchain

For using ARM toolchain , user need to install keil_v5. The process of installing keil_v5 will automatically install ARM toolchain. If keil_v5 is already installed in the root of the C drive. The path of the ARMCC need update in KeilBuild.bat, as shown below.

Figure 2-1. ARM Tool Chain Path

KeilBuild.bat

```
@echo off
@echo %PATH% | findstr /c:"%~dp0tools/msys64/usr/bin">nul
@if %errorlevel% equ 1 set PATH=%~dp0tools/msys64/usr/bin;%PATH%
rem @set PATH=%~dp0tools/msys64/usr/bin;%PATH%

set PROJECT_NAME=at_command
set BOARD_NAME=ec616_0h00


set KEILCC_PATH="C:/Keil_v5/ARM/ARMCC/bin/"
set PROTOCOL_TOOL_DB_PATH="..\PROTOCOL\Tool\UniLogPsParser\DB"
set COMMON_TOOL_DB_PATH="..\TOOLS\UniLogParser\DB"

echo build.bat version 20180330
echo KEILCC_PATH: %KEILCC_PATH%

if not %KEILCC_PATH% == "" (
   if not exist %KEILCC_PATH% (
      echo ERROR: Please check KEILCC_PATH setting, exit!!!
      goto end
   )
) else (
  echo ERROR: Please set KEILCC_PATH firstly, exit!!!
  goto end
)
```

2.1.2 Compilation Step

1. Double click cmd.exe in the root directory and enter the Command Prompt. Shown in figure2-2.
2. Run KeilBuild bat, this batch file supports different parameters, and the parameter is optional. If the user run KeilBuild.bat without

parameter, default project will be compiled. Otherwise, the specific project will compile according to specified parameter.

The format of parameter is: #KeilBuild.bat [board name]-[project name]-[option]

Take board ec616 as an example, table2-1 shows all supported parameters.

Figure 2-2. Command Prompt

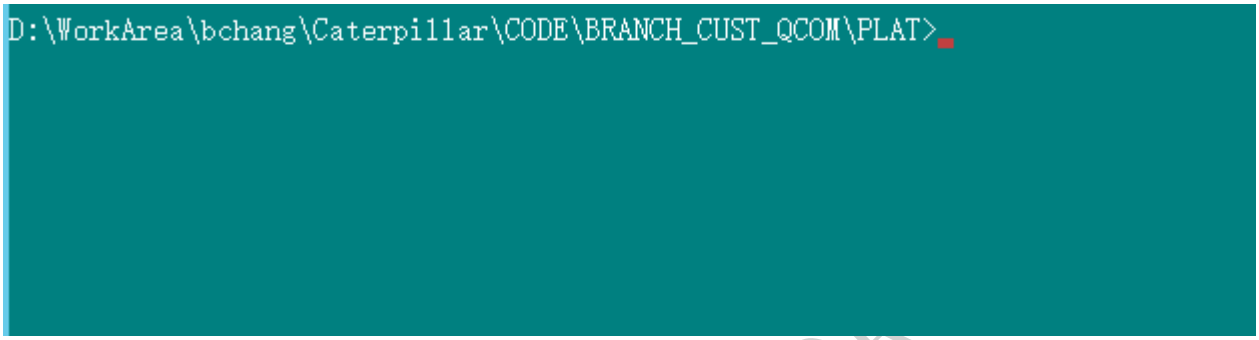


Table2-1 All Support Parameters

parameter	content	description	
Board name	ec616_0h00	EC616 board name	
Project name	at_command	Example project of AT command	Default project
	bootloader	Example project of bootloader	
	driver_example	Example project of driver	
	fs_example	Example project of file system	
	onenet_example	Example project of OneNET	
	Aliyun_example	Example project of aliyun	
	socket example	Example project of socket	
	mqtt_example	Example project of MQTT	
	coap_example	Example project of CoAP	
	lwm2m_example	Example project of LWM2M	
Option	clean	clean current build project	
	clall	clean all projects have built	
	unilog	before compile create debug_log.h	
	doc	create document	
	verbose	output detailed help information during compilation	
	help	output help information	
	list	list all board name and project name support	
	allprojects	compile all the projects	
	defaultall	compile default project and bootloader project	

Use Keilbuild.bat help to check all supported command parameters and usage, as shown in figure2-3.

Figure 2-3. KeilBuild.bat help

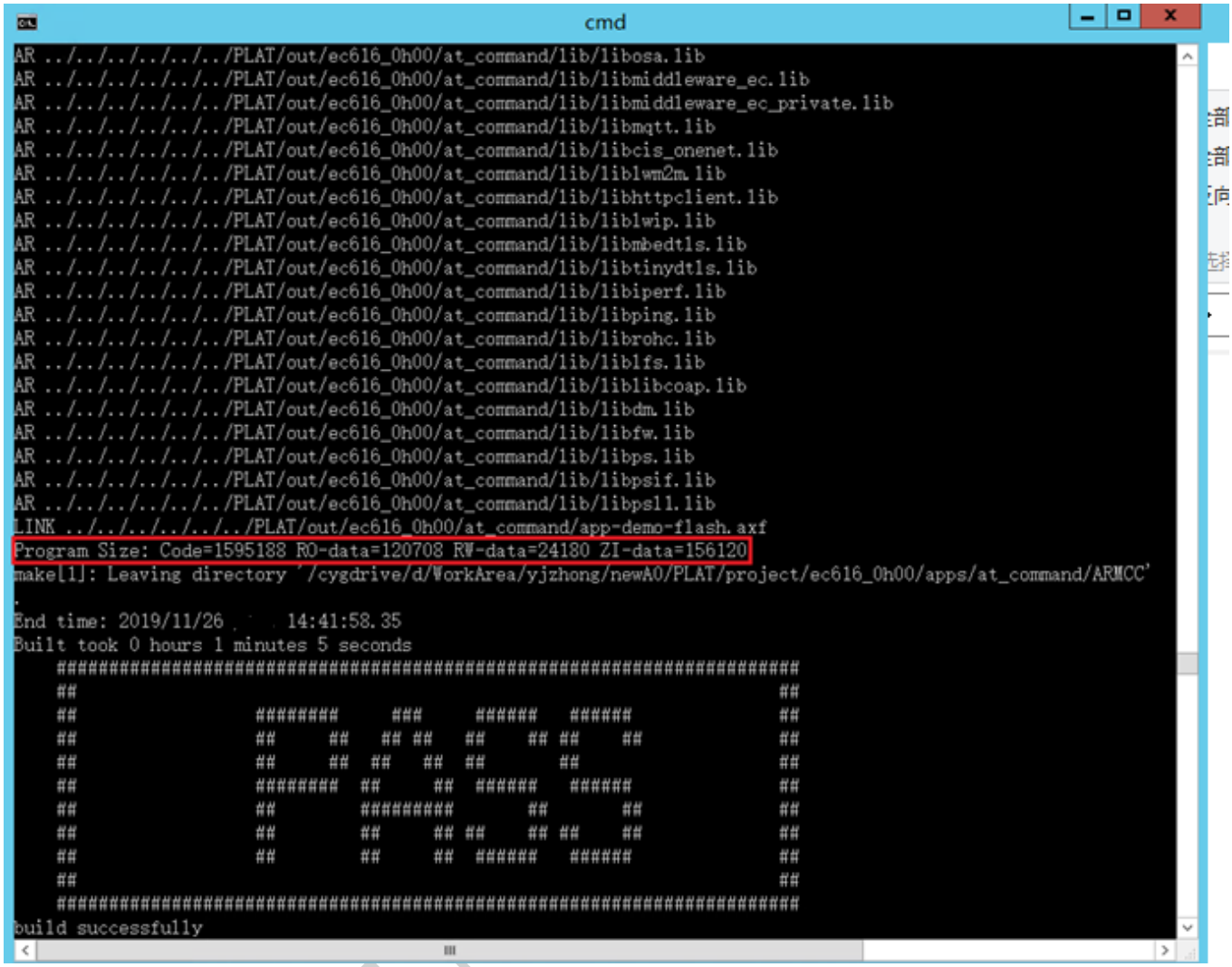
```
D:\WorkArea\yjzhong\newA0\PLAT>Keil_Build.bat help
build.bat version 20180330
KEILCC_PATH: "C:/Keil_v5/ARM/ARMCC/bin/"
OPTION: help
"=============================================================================================="
"                                                                                              "

"--------------------------Show How to Build Project-------------------------------------"
"                                                                                              "

"=============================================================================================="
"                                                                                              "
"                                                                                              "
"                         Keilbuild.bat Board-Project-Option                                   "
"Board,Project,Option could be omitted, if so default Board and project will be used          "
"                                                                                              "
"                                                                                              "
"**********************************************************************************************"
"Supported Options:"
"       doc----run doxygen to generate documents                                               "
"       unilog----pre-parse the log before compile                                             "
"       clean----clean the build output files                                                  "
"       clall----clean all output files for every board and project                            "
"       jekins_release----trigger jekins_release                                               "
"       allprojects----trigger compile all example projects under default board                "
"       list----list current supported boards and supported project for each board             "
"       help---show how to use the build script                                                "
"                                                                                              "
"                                                                                              "
"       Example start -- Example start -- Example start -- Example start                       "
"----------------------------------------------------------------------------------------------"
"       Keilbuild.bat  ec616_0h00-driver_example"
"       build driver_example project for board ec616_0h00"
"**********************************************************************************************"
"       Keilbuild.bat  ec616_0h00-bootloader"
"       build bootloader project for board ec616_0h00"
"**********************************************************************************************"
"       Keilbuild.bat  ec616_0h00-driver_example-clean"
"       clean driver_example project for board ec616_0h00"
"**********************************************************************************************"
"       Keilbuild.bat clean"
"       clean default project for default board
"**********************************************************************************************"
"       Keilbuild.bat clall"
"       clean all output files for every board and project
"**********************************************************************************************"
"       Keilbuild.bat unilog"
"       perform log preparse of default project for default board
"**********************************************************************************************"
"       Keilbuild.bat list"
"       list current supported boards and supported project for each board
"----------------------------------------------------------------------------------------------"
"       Example end -- Example end -- Example end  --  Example end                             "
D:\WorkArea\yjzhong\newA0\PLAT>
```

After compilation, the terminal will output the compilation result. Program size indicates the memory usage.

Figure 2-4. Compile Result

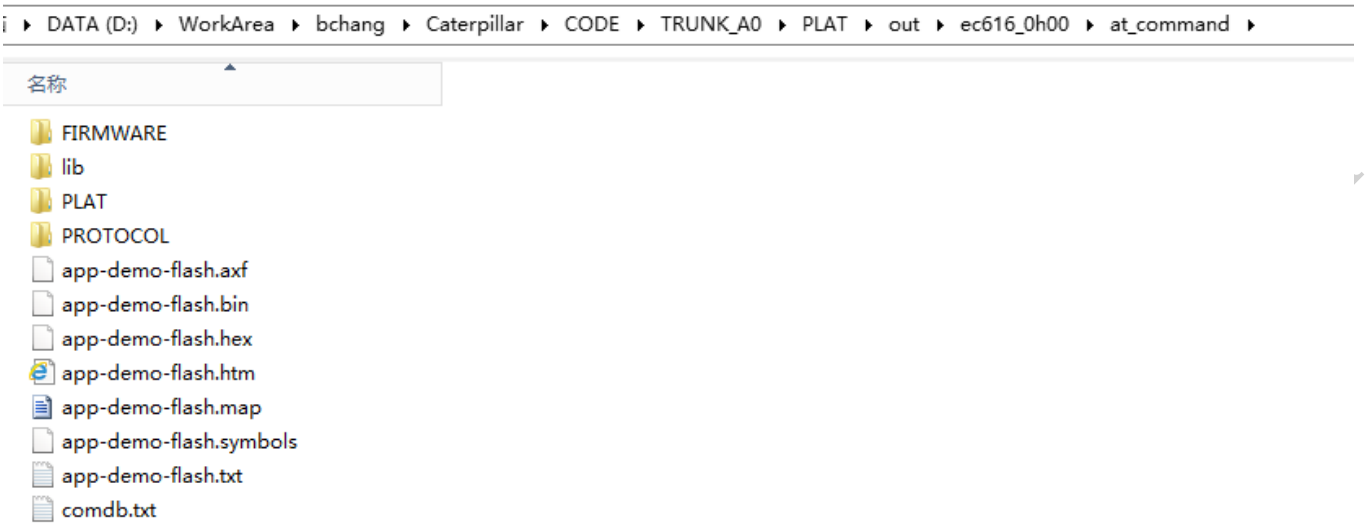


The intermediate files, library files, axf/hex files, map files, and corresponding disassembly files will be output to different subdirectories according to different board and project configurations, as shown below.

Figure 2-5. Out Directory

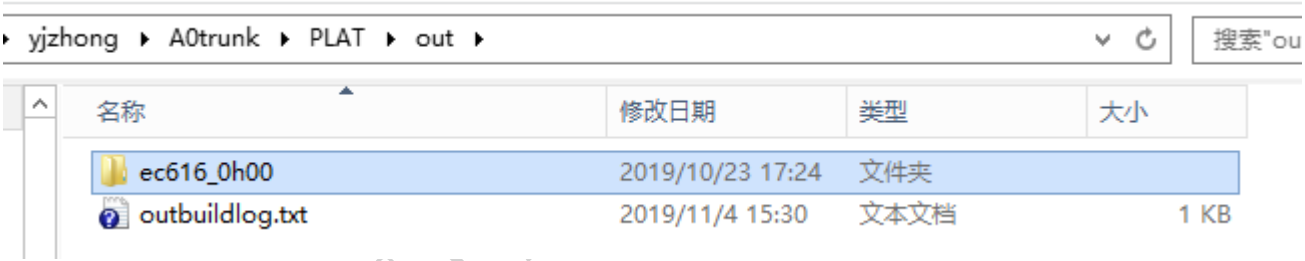
Figure 1. Out directory

Figure 2-6. Output Files



The log of the compilation process is outbuildinglog.txt in the SDK/out, as shown below.

Figure 2-7. Compilation Log



2.2 Create Own Project

Two methods are provided to create your own project.

2.2.1 Use Demo Project

If new added project is similar as some demo project in the SDK, it is suggested to copy and modify the demo project for new project developing. For example, if costumer want to develop project with AT command interaction, at command demo project could be used; and if customer want to develop lwm2m access project, lwm2m demo project could be used.

Take at command project as an example.

Step 1: copy at command example to project/ ec61x_0h00/apps and rename, e.g. my_project

Step 2: add my_project compiling support in keilbuild.bat.

Figure 2-8. Add Project

```
echo PROJECT_NAME: %PARAMETERS% | findstr "mqtt_example"
if not errorlevel 1 (
  set PROJECT_NAME=mqtt_example
)

echo PROJECT_NAME: %PARAMETERS% | findstr "coap_example"
if not errorlevel 1 (
  set PROJECT_NAME=coap_example
)

echo PROJECT_NAME: %PARAMETERS% | findstr "dm_example"
if not errorlevel 1 (
  set PROJECT_NAME=dm_example
)

echo PROJECT_NAME: %PARAMETERS% | findstr "my_project"
if not errorlevel 1 (
  set PROJECT_NAME=my_project
)
```

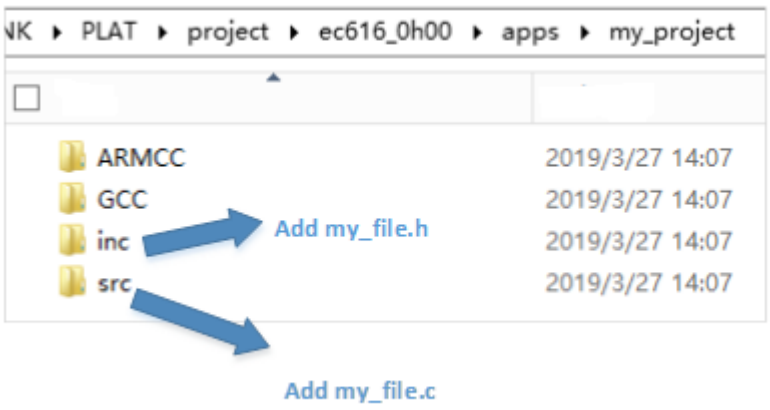
Step 3: customize the makefile to select which module should be built into my_project.

The makefile is located in project/ /ec61x_0h00/ apps/my_project/ARMCC/ makefile. For example, below setting is: enable AT feature, but disable AT DEBUG feature.

BUILD_AT = y
BUILD_AT_DEBUG = n

Step 4: add your own source files. Header file could be added into inc folder, and c file could be added into src files. Shown in figure 2-9

Figure 2-9. Add Source Files



Then modify makefile to support new added files, shown in figure2-10

Figure 2-10. Add Compile Support

```
AVAILABLE_TARGETS = ec616_0h00 ec616z0_0h00
TOOLCHAIN         = ARMCC
BINNAME           = app-demo-flash
TOP               := ../../../../../..

CFLAGS_INC        +=  -I ../inc
obj-y             += PLAT/project/$(TARGET)/apps/at_command/src/app.o \
                     PLAT/project/$(TARGET)/apps/at_command/src/bsp_custom.o \
                     PLAT/project/$(TARGET)/apps/at_command/src/user_config.o\
                      PLAT/project/$(TARGET)/apps/at_command/src/my_file.o\

include $(TOP)/PLAT/tools/scripts/Makefile.rules

ifneq ($(BUILD_AT),y)
$(error This example needs to modify "BUILD_AT" to "y" in device\eigencomm\board\$(TARGET)\ec616_0h00.mk)
endif
```

2.2.2 Use Template Project

project/ec61x_0h00/templates also could be used to develop new project.it is similar as using demo project. But the template is an empty project. If new project is similar to one of the demo project, it is suggested to use the demo project.

2.3 Flash Download and Debug

Download via KEIL IDE and flash tool are supported for image downloading. If you want debug via KEIL IDE, you may use KEIL to download and debug. If only want to download the image and debug with log, flash tool will be used.

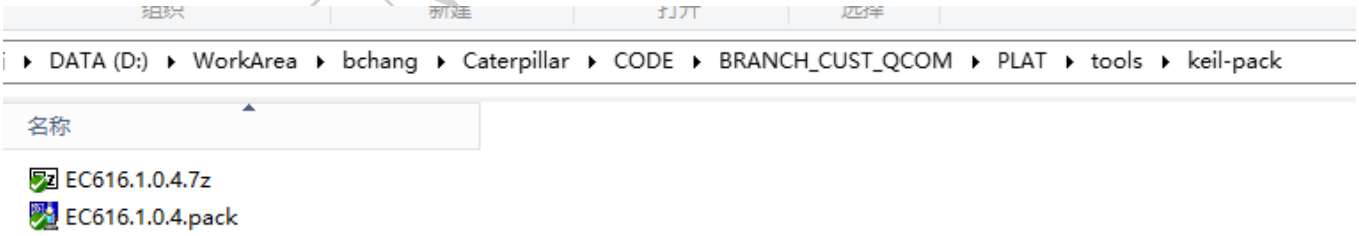
2.3.1 Download with KEIL

If debug using KEIL IDE is necessary, such as step and set breakpoint, this download method is suitable. Keil pack should be setup before download.

This section takes EC616 as an example.

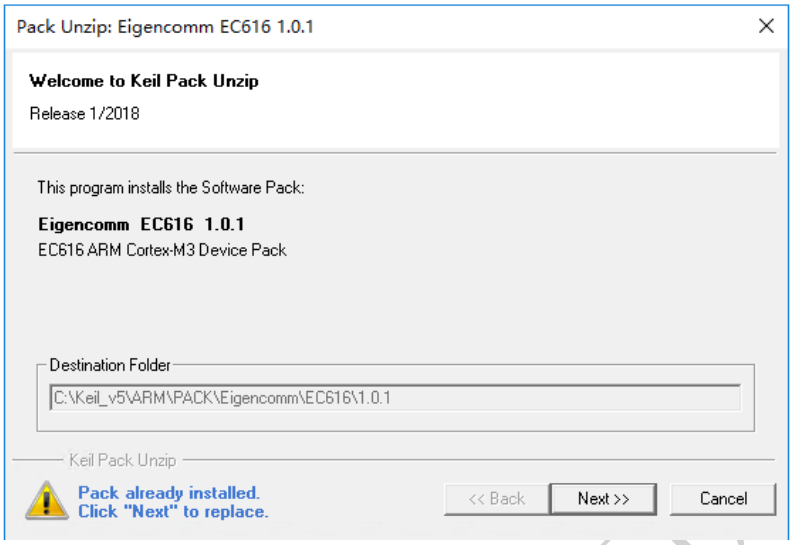
The keil pack is located in tools/keil-pack, double click will trigger the setup.

Figure 2-11. Keil pack location



Step 1：set up keil pack, click next to go on.

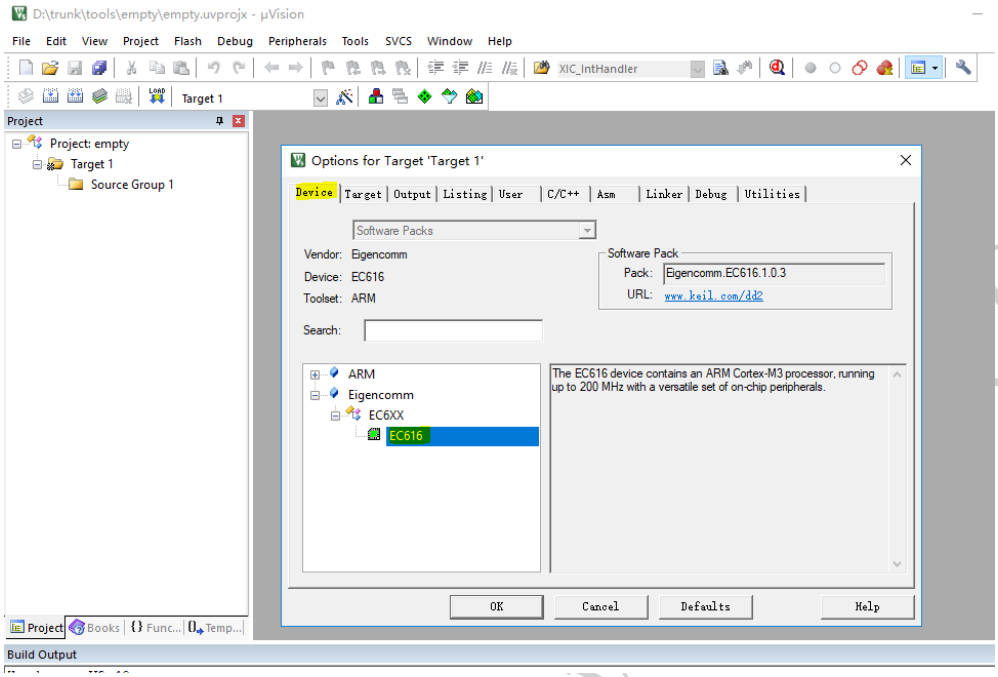
Figure 2-12. Keil pack Setup



Step 2：download setting

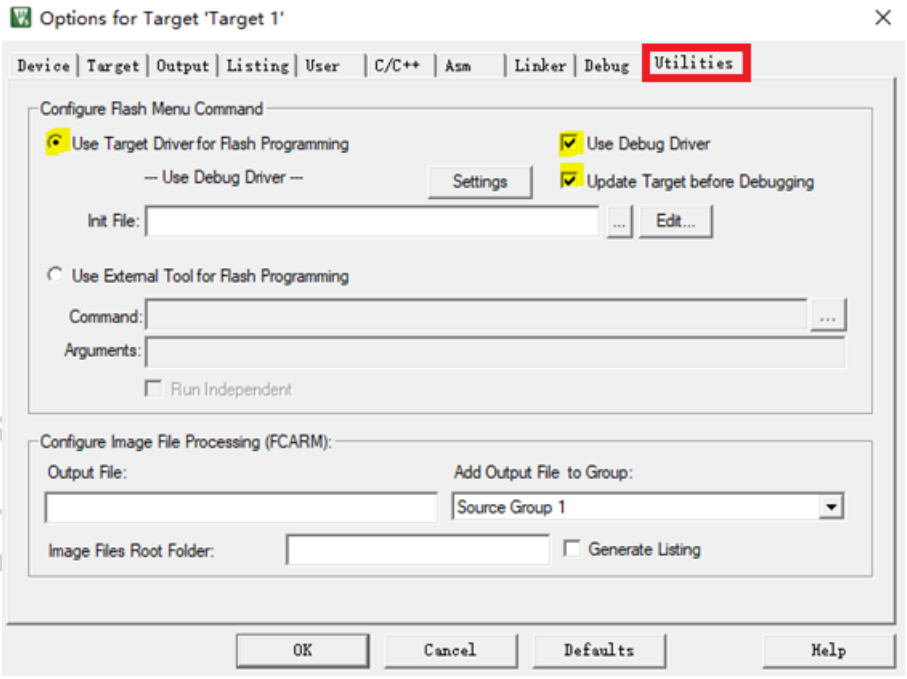
1） Select ec616 in figure2-13

Figure 2-13. Select device



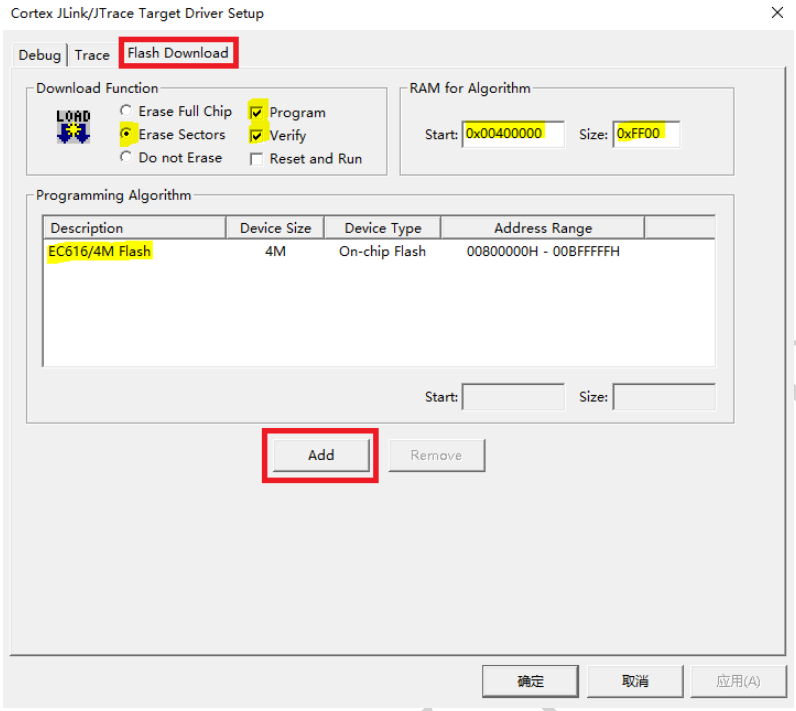
2） Setting as figure2-14 in utilities blank，and click settings to enter flash blank.

Figure 2-14. Utilities setting



3） Add flash setting as shown in figure2-15

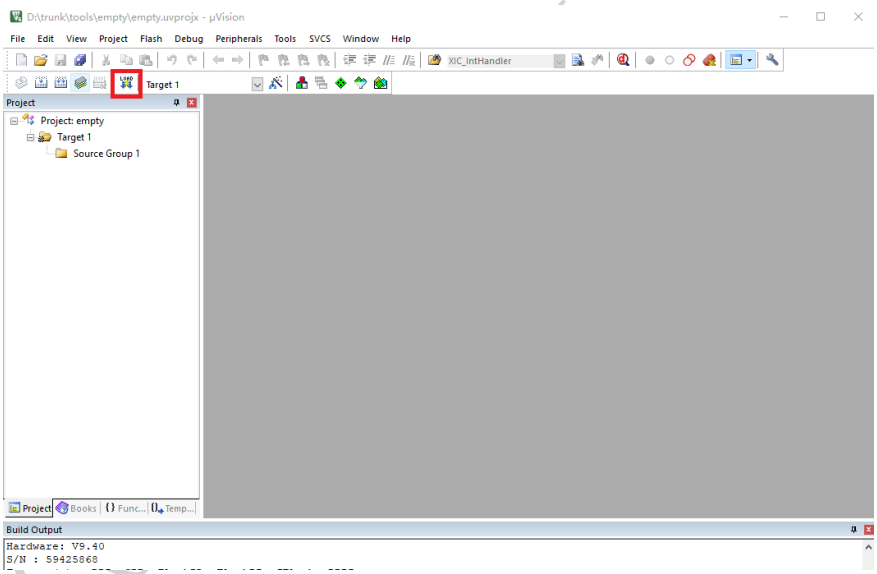
Figure 2-15. Flash Setting



Step 3: trigger download.

Click load icon in figure2-16 to trigger download.

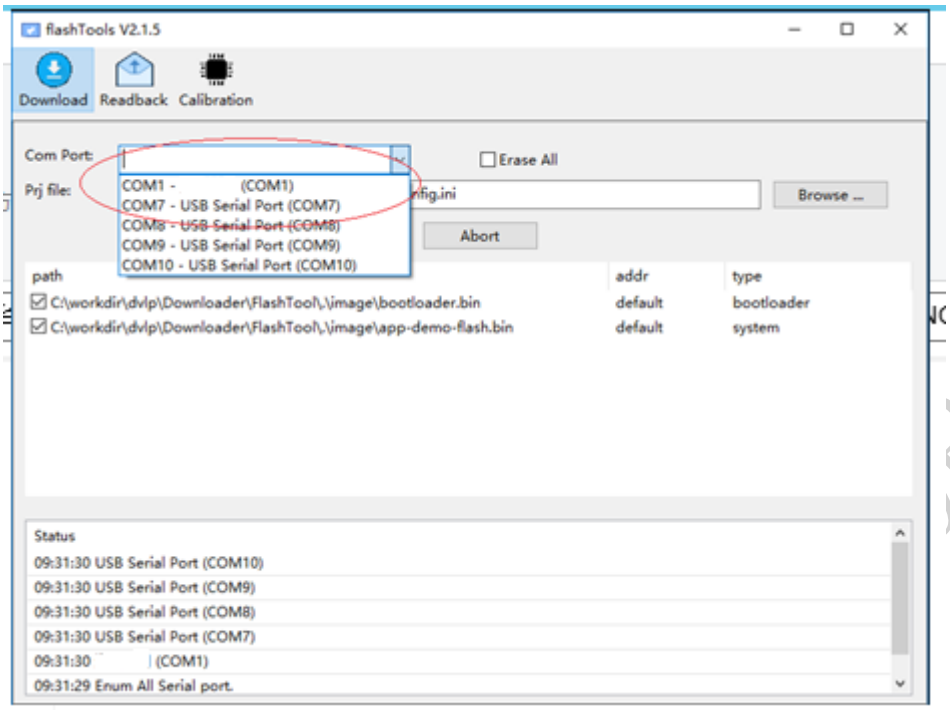
Figure 2-16. Trigger download



2.3.2 Download Using Flash Tool

The flashtool.exe is developed by eigencomm, shown in figure2-17 and will be released to customer. There is a document inside the flashTool pack.The detail operation description is in the doc FlashTools user guide.

Figure 2-17. Flash Tool



2.4 Debug with EPAT

EPAT is a diagnostic tool developed by eigencomm. All logs will be output and shown in EPAT for analysis.

It will be released to customer and there is a detail operation document inside the EPAT pack, EPAT User Guide.

3. Version

Version	Date	Comments
Draft	2018-11-11	Draft
V1.0	2020-02-09	English version
V1.1	2020-04-09	Add more section

4. About US

Shanghai EigenCOMM Technology Co.,Ltd is established in Feb 2017 in Zhangjiang Hi-Tech park, Shanghai China (www.eigencomm.com) . EigenCOMM focuses on cellular based IoT chipset and solution. With accumulation of experience over the years in algorithm, L1/L2/L3 protocol, SoC, RF , platform & application software as well as the communication architecture and system, team makes its way to IoT industry.

The ultra-low power NB-IoT chipset EC616(S) has already enter mass production stage. The 4G Cat.1 chipset EC618 is in engineering samples stage, 5G is also in the R&D stage.

Eigencomm Technoligies Ltd.

Address：Room 707, No 298 Xiangke Road, Shanghai, China

Zip Code：201210

Phone：

E-mail：

Website：http://www.eigencomm.com

Support Window

E-Mail：support@eigencomm.com